



From guidelines to practice: assessing Android app developer compliance with google's security recommendations

Shishuai Yang¹ · Qinsheng Hou^{1,2} · Shuang Li¹ · Fenghao Xu³ · Wenrui Diao¹ 

Accepted: 30 September 2024 / Published online: 28 October 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

The popularity of Android OS is largely credited to massive number of apps, and many app developers are involved in this ecosystem. On the other hand, various vulnerabilities are introduced into apps by developers carelessly, bringing security risks to users. To facilitate secure development and avoid common API misuses, Google provides a series of security guidelines and development practices for developers on official developer community websites. However, the adoption rate of these security guidelines in the real-world has not been systematically evaluated. In this work, through large-scale app measurement (108,091 apps from Google Play) and analysis, we investigated whether app developers follow the official Android security guidelines and the possible reasons behind it. In practice, we selected nine guidelines and mapped them to four OWASP MASVS control groups (MASVS-STORAGE, MASVS-NETWORK, MASVS-PLATFORM, and MASVS-CODE) as representatives, covering: (1) sensitive data storage; (2) validation check for file paths; (3) network security measures; (4) custom permission protection; (5) webview objects usage; (6) intent vulnerability; (7) secure file creation modes; (8) hardware ID usage; (9) man-in-the-middle attacks. We also designed the corresponding detection strategies to identify violations of the guidelines. The results show that most developers (> 90%) comply with Guidelines 1 and 7. However, some guidelines have not been followed properly. For Guidelines 2, 3, 4, 5, 6, and 8, less than 60% of developers followed Google security suggestions.

Communicated by: Federica Sarro

✉ Wenrui Diao
diaowenrui@link.cuhk.edu.hk

Shishuai Yang
shishuai@mail.sdu.edu.cn

Qinsheng Hou
houqinsheng@mail.sdu.edu.cn

Shuang Li
lishuang1024@mail.sdu.edu.cn

Fenghao Xu
fhxu@seu.edu.cn

¹ School of Cyber Science and Technology, Shandong University, Qingdao, China

² QI-ANXIN Technology Research Institute, Beijing, China

³ School of Cyber Science and Engineering, Southeast University, Nanjing, China

Keywords Android OS · Android developers · App vulnerabilities · Google Play · Security guidelines

1 Introduction

As the most popular mobile operating system, Android has attracted an increasing number of developers to join the app development community. The number of mobile developers is expected to reach 27.7 million by 2023 (Simplilearn 2022). Furthermore, due to the support of the Android SDK and the official development documentation, the number of apps in the Android ecosystem has increased significantly. As of 2024, there are 3.95 million apps available on Google Play (Turner 2024). However, the lack of security awareness among developers has led to various app security vulnerabilities, as they tend to prioritize app functionality over security requirements (Gao et al. 2021). Disturbingly, recent data from the Atlas VPN team indicates that 63% of Android apps contain known security vulnerabilities, averaging 39 vulnerabilities per app (C. 2021).

To further improve app security and avoid common API misuses, Google proposes *app security best practices* in the Android Developer Documentation to guide a wide range of developers (Google 2024d). These guidelines cover various aspects of app security, such as data storage, communication, permission, and user privacy. For example, the guideline suggests developers should only use the log functions to output debugging information, not containing sensitive data. Though Google provides many app security guidelines, their adoption rate in the real-world has not been systematically evaluated. The essential research questions have not been systematically answered, such as "*Do app developers comply with these guidelines? If not, what are the corresponding reasons?*"

Our work In this work, through large-scale app measurement and analysis, we investigated guidelines' deployment status and possible reasons of guideline violations among developers. On a high level, we need to select appropriate guidelines, design & implement detection rules, analyze the measurement results, and evaluate the guidelines' adoption rate.

Due to the large number of app security guidelines provided by Google and the continuous addition of new security guidelines, we cannot cover all the security guidelines shown in the official documentation in this work. We selected nine security guidelines based on OWASP MASVS (Mobile Application Security Verification Standard) (OWASP 2024) and mapped nine security guidelines to four MASVS control groups: [MASVS-STORAGE: Guideline 1 and 2]; [MASVS-NETWORK: Guideline 3 and 4]; [MASVS-PLATFORM: Guideline 4, 5 and 6]; [MASVS-CODE: Guideline 7, 8 and 9]. More details are given in Section 2.2.

MASVS-STORAGE: Storage

Guideline 1: *Do not leak sensitive data on external storage or in logs* (Google 2024j, p. o).

Guideline 2: *Check the validity of file paths when reading files* (Google 2023e, b).

MASVS-NETWORK: Network Communication

Guideline 3: *Apply network security measures* (Google 2024a).

MASVS-PLATFORM: Platform Interaction

Guideline 4: *Exported components should be protected by appropriate custom permissions* (Google 2024l, f, 2023a).

Guideline 5: *Disable JavaScript support and local file loading when using WebView objects* (Google 2024q).

Guideline 6: *Avoid introducing intent vulnerabilities* (Google 2023c, 2024g, 2023d).

MASVS-CODE: Code Quality

Guideline 7: *Use secure modes when creating files* (Google 2024m).

Guideline 8: *Avoid using non-resettable hardware IDs* (Google 2024k).

Guideline 9: *Improving app code quality to mitigate man-in-the-middle attacks* (Google 2024b, 2023f, g).

To evaluate the deployment status of the above security guidelines among Android app developers, we constructed the corresponding lightweight detection rules. Further, we conducted a large-scale measurement on 108,091 apps from Google Play. In addition, we combined multiple dynamic and static analysis methods to obtain possible reasons for guideline violations, such as measurement results analysis, manual reverse analysis, development simulation analysis, and community resources analysis. By cross-validating the results obtained by different analysis methods, we can infer the possible reasons for guideline violation.

The measurement results show that most developers (> 90%) comply with Guidelines 1 and 7, while the situations of other guidelines are not very well. For Guidelines 2, 3, 4, 5, 6, and 8, less than 60% of developers followed them well. Although the specific reasons are different by guidelines, in general, both Google (official developer documentation and Android Studio – the official IDE) and app developers (code development habits) should be responsible for the violation of Guidelines 2, 3, 4, 5, 6, 8, and 9.

Contributions The main contributions of this paper are:

- **Guideline Deployment Evaluation.** Based on 108,901 apps collected from Google Play, we evaluated the deployment status of security guidelines in the wild.
- **Violation Reason Analysis.** We systematically investigated the possible reasons for guideline violations in apps, which can help improve the adoption rate of guidelines.
- **Comprehensive Measurement Study.** In the conference version of this work, we evaluated the compliance status of third-party market apps with security guidelines. For the extended version, we expanded our analysis to evaluate the compliance status of Google Play apps with more security guidelines.
- **Practical Results.** This work can help developers understand the security risks and vulnerabilities in apps and promptly correct them, thereby enhancing user data's security and privacy protection. Also, this work can help Google understand the reasons why guidelines are not well follow, and design more reasonable guidelines.

Remarks This work is an extension of our conference version appearing in Proceedings of APSEC 2023 (Yang et al. 2023). In the conference version of this work, we evaluated the compliance status of third-party market apps with security guidelines. In this extension, we constructed the Google Play apps dataset and updated the measurement results, images, and tables for Guidelines 1, 2, 4, 7, and 8 based on the latest dataset. In addition, we reviewed the Android security documentation to cover more guidelines, including Guidelines 3, 4,

6, and 9. We redesigned the detection rules for these newly added guidelines, evaluated the compliance status of these guidelines, and investigated the possible reasons for guidelines violations.

2 Android security and guidelines

In this section, we provide the necessary background of Android OS security mechanisms and Google's security guidelines for app developers.

2.1 Android security mechanisms

On Android, Google designs a series of security measures to protect data, such as permission and sandbox mechanism.

App sandbox In Android, apps run in independent sandboxes. Each Android app runs in its own process, possessing independent memory space and private data (Google 2024e). The sandbox can be treated as a safe execution environment for apps and a restriction for malicious attackers. The implementation of the sandbox mechanism is based on the Linux access control mechanism, which utilizes user IDs to separate different app processes. By default, apps cannot interact with each other, and data access to other apps is restricted. The deployment of the sandbox mechanism further clarifies the app's security boundary and minimizes the impact of malicious behaviors.

Permission Android refines the access control mechanism of Linux and uses a permission-based mechanism to protect access to sensitive resources (Google 2024i). System permissions are usually used to protect system resources. For example, if apps want to use the camera, apps need to request the CAMERA permission. Otherwise, the system will prohibit apps from accessing the camera. Android permissions are divided into three levels (Google 2024c): (1) Normal: As long as the app applies for the permission, it will be granted automatically. (2) Dangerous: The app applies for and requires user authorization to use the permission. (3) Signature: If the requesting app is signed with the same certificate as the app declaring the permission, it can be used.

2.2 Google's guidelines for app security

Here we summarize Google's guidelines and suggestions for app security design:

Guideline 1: *Do not leak sensitive data on external storage or in logs* (Google 2024j, p. o).

⇒ The storage space of Android OS is divided into internal storage and external storage. Internal storage is private for apps, while external storage is shared among apps. Therefore, app developers should store sensitive data on internal storage and non-sensitive data on external storage. In addition, the log function is designed to output some debugging information at runtime, and developers may leak sensitive data related to the app through logs. If malicious apps obtain the READ_LOGS permission, they are capable of reading sensitive data leaked by other benign apps via logs.

Impact The leakage of sensitive information to external storage and logs may cause unauthorized access, data leakage, identity theft, financial losses and social engineering attacks. Attackers can take advantage of this leaked information to carry out unauthorized operations, fraudulent transactions and deceptive interactions, thus causing serious harm to users.

Guideline 2: *Check the validity of file paths when reading files* (Google 2023e, b).

⇒ The validity of file paths implies that when developers decompress files or use the `openFile` method under the `ContentProvider` class for file sharing, they are required to check if file paths contain relative paths represented by `".."`. If developers fail to check the validity of file paths, apps might be exposed to the risk of **path traversal vulnerability**. In that case, apps may incorrectly open files indicated by the attacker or decompress the file to the location specified by the attacker.

Impact Path traversal vulnerabilities allow attackers to access, modify, or delete files and data that should be protected, posing severe security risks including unauthorized access, data leakage, remote code execution (RCE), system file corruption, privilege escalation, and service disruption.

Guideline 3: *Apply network security measures* (Google 2024a).

⇒ This guideline mainly includes two meanings:

- Add NSC (network security configuration) file. The NSC file is an XML file, and it allows developers to define the security requirements of apps in network communication. Developers can use the NSC file to specify trusted certificate authorities, configure hostname validation, and define other security settings related to network connections. This helps ensure secure network communication and aligns with the app's security requirements.
- Disable Cleartext Network Traffic. Enabling cleartext network traffic in apps can introduce security risks to apps, allowing attackers to monitor and manipulate transmitted data, such as passwords and credit card numbers. Before API level 28, apps enable global cleartext network traffic by default. From API level 28, apps disable global cleartext network traffic by default. Developers can open cleartext network traffic using two methods: (1) Set the `usesCleartextTraffic` attribute of the `Application` component to `true` to enable global cleartext network traffic. (2) In the NSC file, set the `cleartextTrafficPermitted` attribute of the `<base-config>` tag to `true` to enable global cleartext network traffic, and set the `cleartextTrafficPermitted` attribute of the `<domain-config>` tag to `true` to enable cleartext network traffic for specific domains.

Impact The absence of the aforementioned network security measures can lead to apps communicating with untrusted servers and transmitting data in plaintext, which can result in various security risks including data leakage, man-in-the-middle (MITM) attacks, unauthorized access, and data integrity compromise.

Guideline 4: *Exported components should be protected by appropriate custom permissions* (Google 2024l, f, 2023a).

⇒ In addition to system permissions, permissions also can be defined by app developers, called custom permissions. They are used to protect the data generated by apps and usually act on the four major components of Android. Using custom permissions can achieve data sharing between trusted apps and prevent malicious apps from accessing data.

This guideline focuses on two aspects: (1) Android has four major components: Activity, Service, Receiver, and Provider. To make the four components accessible to other third-party apps, developers can set components to be exported. Further, developers can use the signature-level custom permissions to protect these exported components. The signature-level ensures the data is shared among trusted apps. (2) Developers must follow the principle of least privilege, apply the defined custom permissions to exported components, and do not keep unused permissions in the manifest files.

Impact The exported components being unprotected or not protected by the signature-level custom permissions can bring various security risks to apps, including unauthorized access, data leakage, malicious operations by invoking unprotected components, denial-of-service (DOS) attacks, and privilege escalation. Over-declaring custom permissions increases the risk of apps being maliciously exploited. Attackers may use these permissions to perform unauthorized operations, accessing or modifying sensitive data. In addition, over-declared permissions can be exploited by other apps to conduct privilege escalation attacks.

Guideline 5: *Disable JavaScript support and local file loading when using WebView objects* (Google 2024q).

⇒ WebView is used to embed a browser engine within apps to load and display web content. WebView objects should not allow users to navigate to websites beyond the developer's control. It is advisable to employ the allowlist mechanism whenever possible to restrict the content that WebView objects can load. In addition, never enable JavaScript interface unless developers completely control and trust the content of the app's WebView objects. The `shouldOverrideUrlLoading` callback method of WebView can determine the action to be taken when loading a new URL, such as prohibiting the opening of specific URLs. Therefore, developers can ensure that WebView objects load trusted content by overriding this callback method.

In addition, WebView objects can not only load web content but also be used to load various files. The APIs `setAllowFileAccess`, `setAllowFileAccessFromFileURLs`, and `setAllowUniversalAccessFromFileURLs` are mainly used for setting parameters related to local file loading. They control whether the WebView object is allowed to load local files, whether file URLs are permitted to access the URLs of other files, and whether file URLs are allowed to access content from any source. To enhance the security of apps, these options should be disabled unless there is a genuine need to load content from the local file system within WebView objects.

Impact Enabling JavaScript support in WebView objects without verifying the trustworthiness of the content to be loaded and setting file-loading-related API parameters to true exposes apps to significant security risks in the real world. These risks include data leakage, RCE, XSS attacks, privilege escalation, and the manipulation or deletion of local files. These vulnerabilities severely threaten user privacy, data security, and system integrity.

Guideline 6: *Avoid introducing intent vulnerabilities.* (Google 2023c, 2024g, 2023d).

⇒ Intent serves as a foundational mechanism for communication and interaction among various components. If developers do not use intent correctly, it can lead to a series of security vulnerabilities:

- **Implicit Intent Hijacking.** When an app dispatches an intent without a fully qualified target component name, it opens the door to implicit intent hijacking. This allows a malicious app to register an intent filter to intercept the intent instead of the intended app.
- **Intent Scheme Hijacking.** It refers to potential security vulnerabilities that may arise when processing intent schemes from `Uri`. Attackers can craft `Uri`s with a specific intent scheme to trigger insecure operations, such as launching private components of apps. Apps may be vulnerable to this attack if they call the `Intent.ParseUri` method to parse `Uri`s.
- **Intent Redirection.** Intent redirection vulnerability refers to a situation where malicious apps manipulates Intents, redirecting them to unauthorized or inappropriate components.

Impact Implicit intent hijacking can lead to unauthorized access, where malicious apps intercept implicit intents and obtain sensitive data, resulting in data and privacy leakage. Intent scheme hijacking can cause phishing attacks by displaying fake pages, tampering with data, or deceiving users into interacting with malicious apps. Intent redirection can lead to privilege escalation, where malicious apps redirect intents to gain higher permissions or access restricted components, causing unintended actions or excessive resource consumption, potentially leading to DOS attacks.

Guideline 7: *Use secure modes when creating files* (Google 2024m).

⇒ App developers can select the file creation mode when creating the `SharedPreferences`, database, and private files. `SharedPreferences` file is a lightweight storage class, mainly used to store preference data (key-value pairs). Database files are mainly used to store structured record information, and private files are mainly used to store apps' private data. Each mode corresponds to a constant value. For example, `MODE_PRIVATE` corresponds to `0x0000`.

Insecure file modes pose significant security risks. For example, under `MODE_WORLD_READABLE` and `MODE_WORLD_WRITEABLE`, the created files can be read and written by other apps without any permission. These insecure modes are designed to share files among apps, such as configuration files. Also, `MODE_MULTI_PROCESS` only can be used when creating `SharedPreferences` files. This file mode is mainly designed to launch the multi-process mode to improve operating efficiency and share `SharedPreferences` among multiple processes. However, this mode is not process-safe and may cause inconsistent reading and writing of `SharedPreferences` files by multiple processes. Currently, Google has provided more secure design solutions to achieve the same functions and recommends not using these insecure modes.

Impact Using insecure file modes when creating files can lead to security risks such as data leakage, data tampering, and privilege escalation. This allows other apps to access or modify sensitive information and may even inject malicious code, thereby threatening user privacy and device security.

Guideline 8: *Avoid using non-resettable hardware IDs* (Google 2024k).

⇒ There are four hardware identifiers in Android, including IMEI (International Mobile Equipment Identity), S/N (Serial Number), MAC Address, and SSAID (Android ID). The

hardware identifier is mainly used for device binding. These hardware identifiers may be leaked to attackers through network transmission, logs, etc.

Impact Hardware identifiers can be used to track users, revealing their location and behavior patterns. Moreover, improper use of these data can infringe on users' privacy rights, and they can be used for market analysis or targeted advertising. If hardware identifiers fall into the hands of malicious third parties, it could lead to identity theft or fraud.

Guideline 9: *Improving app code quality to mitigate man-in-the-middle attacks* (Google 2024b, 2023f, g).

⇒ If apps fail to properly implement hostname verification or server certificate validation in SSL/TLS communications, they become vulnerable to MITM attacks. This vulnerability manifests in three primary forms:

- **WebView SSL Error.** Developers may encounter certificate validation errors while using WebView objects, but developers ignore certificate validation errors.
- **Unsafe Hostname Verification.** An unsafe hostname verification refers to apps not properly verifying the server's hostname, accepting all hostnames in two ways: (1) **Allow All Hostnames.** Use `ALLOW_ALL_HOSTNAME_VERIFIER` field. (2) **Insecure Verification.** The `Verify` method of the `HostnameVerifier` interface is not implemented correctly.
- **Unsafe X.509 TrustManager.** An insecure `X509TrustManager` implementation indicates that apps do not properly verify the server's certificate it is communicating with.

Impact These vulnerabilities allow attackers to intercept and tamper with communication content, accept invalid or malicious server certificates, connect to fake servers, steal sensitive data, hijack user sessions, or launch DOS attacks, severely affecting the security of apps and the privacy of user data.

3 Methodology and dataset

To evaluate the proposed app security guidelines, in this section, we illustrate our measurement approaches and constructed app datasets.

3.1 Methodology

As illustrated in Fig. 1, on a high level, our measurement contains three main steps, as follows:

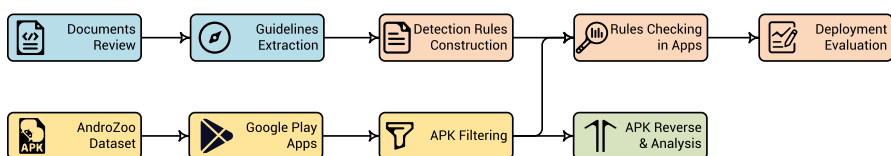


Fig. 1 Workflow of our measurement

- *Guidelines Selection.* First, we reviewed the entire Android security documentation to extract suggestions on app security, as described in Section 3.2.
- *Dataset Construction.* Next, we constructed the app dataset used in the study based on AndroZoo (Allix et al. 2016), as described in Section 3.3.
- *Measurement and Analysis.* Finally, we evaluated the deployment status of security guidelines and investigated the possible reasons of guideline violation, as described in Section 3.4.

3.2 Guidelines selection

Google’s suggestions for enhancing Android app security span a comprehensive suite of guidelines, addressing critical aspects such as interaction, data, and network. Furthermore, Google routinely updates the official Android documentation with fresh security suggestions to address new challenges. Given these factors, our analysis in this work is limited to evaluating a portion of the security guidelines presented in the official Android documentation. We select guidelines based on OWASP MASVS (OWASP 2024), which is the industry standard for mobile app security. The OWASP MASVS is endorsed by numerous stakeholders, including prominent entities like Android, CREST, and NIST. The standard is categorized into different groups of security controls, identified as MASVS-XXXXX, representing the most critical areas of the mobile attack surface. The MASVS control groups consist of a total of eight control groups. We selected nine security guidelines from the Android documentation and mapped them to four MASVS control groups, namely MASVS-STORAGE, MASVS-NETWORK, MASVS-PLATFORM, and MASVS-CODE.

Our research covers four of the eight MASVS control groups because these groups address more fundamental and widespread security issues. Addressing them can significantly reduce the overall security risks of apps, providing users with a safer usage environment. For example, insecure data storage can lead to the leakage of sensitive information, which is a common risk in mobile apps. Furthermore, Google has clearly indicated in its official documentation (Google 2024h) that MASVS-STORAGE, MASVS-NETWORK, MASVS-PLATFORM, MASVS-CODE, and MASVS-CRYPTO are security risks that need to be mitigated as a priority. Our research does not include the security risks under the MASVS-CRYPTO group because the misuses of cryptography have already been extensively investigated by the research community.

Note that some security suggestions focus on relevant development aspects. For better demonstration, we aggregated multiple practices/suggestions into one guideline. After aggregation, we obtained 9 security guidelines, including 20 suggestions on app security, as shown in Section 2.2.

3.3 Dataset construction

Google Play, serving as the official app store, has implemented more stringent review processes than third-party markets. Nevertheless, whether apps on Google Play better adhere to Google’s security guidelines and thus offer higher security compared to apps on third-party markets remains an open question. Consequently, a comprehensive analysis of apps sourced directly from Google Play is crucial to accurately assess their compliance with Google’s security guidelines. Since Google Play did not provide the app bulk downloading APIs anymore, we used AndroZoo to construct the dataset used in this study. AndroZoo is a growing collection of Android apps collected from multiple app markets, and we mainly obtain apps

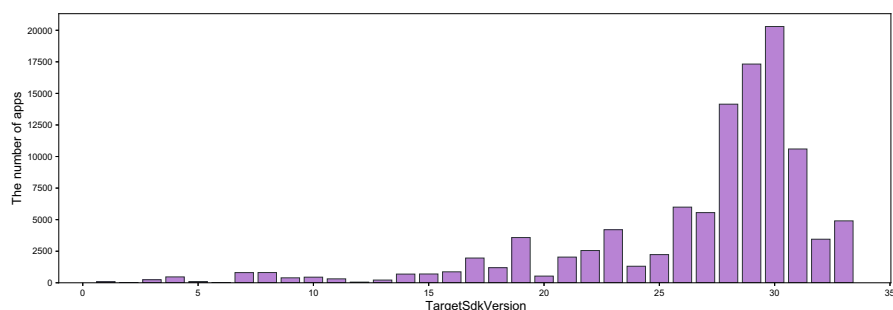


Fig. 2 Distribution of app targetSdkVersion

published on Google Play from AndroZoo. Based on the SHA256 value of apps, we ensure the uniqueness of apps in the dataset. In addition, we removed some corrupted apps and finally got 108,091 apps.

The distribution of the targetSdkVersion of apps is shown in Fig. 2, covering targetSdkVersion 1 to the latest targetSdkVersion 33 (Android 13), and 82.66% of apps with targetSdkVersion ≥ 23 (Android 6.0). To further analyze the characteristics of our dataset, we also examined the distribution of app categories. Since AndroZoo does not collect app category information when collecting APK files, we use google-play-scraper (JoMingyu 2024) to retrieve the app category information from Google Play. Due to the fact that Google frequently removes apps from Google Play that violate policies or pose risks to users and the overall Android ecosystem, and developers also often take down apps for personal reasons. Therefore, out of the 108,091 apps in the dataset, we only collected category information for 49,065 apps. The proportions of each category are shown in Table 1.

Table 1 Category distribution

Category	Percentage	Category	Percentage
GAME	12.66%	COMMUNICATION	2.15%
EDUCATION	12.00%	NEWS_AND_MAGAZINES	2.14%
BUSINESS	7.28%	MAPS_AND_NAVIGATION	1.84%
TOOLS	6.29%	MEDICAL	1.61%
FINANCE	5.98%	PHOTOGRAPHY	1.29%
HEALTH_AND_FITNESS	5.82%	AUTO_AND_VEHICLES	0.96%
LIFESTYLE	5.15%	HOUSE_AND_HOME	0.71%
PRODUCTIVITY	4.54%	VIDEO_PLAYERS	0.69%
SHOPPING	4.09%	WEATHER	0.56%
TRAVEL_AND_LOCAL	3.77%	ART_AND_DESIGN	0.50%
ENTERTAINMENT	3.48%	DATING	0.41%
MUSIC_AND_AUDIO	3.07%	PARENTING	0.32%
BOOKS_AND_REFERENCE	2.84%	BEAUTY	0.28%
FOOD_AND_DRINK	2.38%	EVENTS	0.27%
SPORTS	2.29%	LIBRARIES_AND_DEMO	0.13%
SOCIAL	2.23%	COMICS	0.09%
PERSONALIZATION	2.20%		

3.4 Measurement and analysis

After extracting the security guidelines, we implemented all the development behaviors described by the guidelines, taking into account the different implementations of these guidelines in apps, especially for guideline violation. According to the implementation form of guidelines in apps, we heuristically constructed detection rules for each guideline and performed large-scale measurements on the apps in our dataset. During the build process, apps are required to include certain official libraries. These official libraries are usually supporting libraries that keep apps running, and their package names usually start with `com.google.*`, `com.android.*`, `androidx.*`, and `android.*`. These official libraries are not introduced based on the developers' subjective intent. Therefore, to make the measurement results more accurate, we did not evaluate the compliance of some official libraries to these guidelines. After that, we conducted a multi-dimensional analysis of the measurement results to evaluate the deployment of security guidelines proposed by Google.

After obtaining the measurement results, we employ various methods to analyze the possible reasons for guideline violations, as shown below:

- *Measurement results analysis*: Based on our measurement results of each guideline, we can directly infer the possible reasons of guideline violation.
- *Manual reverse analysis*: We unpacked apps that violated this guideline and analyzed the code. In practice, we focused on apps that violated this guideline multiple times, apps in different categories and versions.
- *Development simulation analysis*: We developed a demo app to re-produce the process of misuse and analyzed the possible reasons of guideline violation.
- *Community resources analysis*: Developers usually learn the use of API from some third-party Q&A websites, so we checked some online Q&A sites (such as Stack Overflow) to find hints of guideline violation.

4 Measurement and findings

In this section, we present our empirical research methods and results on the app security guidelines described in Section 2.2.

Guideline 1: Do not leak sensitive data on external storage or in logs.

In this guideline, we mainly focus on whether developers leak some sensitive data on external storage or in logs.

Leakage on external storage We extracted the filenames on external and internal storage according to the file storage paths in apps. To characterize how developers use the external storage, we performed a word frequency analysis on the filenames extracted from external storage. Also, third-party libraries usually create files to save some information. To exclude the influence of filenames generated by third-party libraries on the frequency of hot words, we save each filename into a collection for deduplication. In addition, we removed some meaningless words, such as “android” and “app”.

The word frequency analysis results show that developers mainly use external storage to save pictures (such as “photo”, “image”, “camera”, “wallpaper”), videos (such as “video”, “audio”), and other media data. We also compared whether there is an overlap between external and internal storage hot words. The result shows that 72 of the top 200 hot words on internal and external storage are the same. It means that developers cannot clearly distinguish whether some types of files should be stored on internal or external storage. As a result,

Table 2 Log levels configured in apps

Log Level	Verbose	Debug	Info	Warning	Error
Amount	768	1,483	1,217	791	942
Percentage	14.77%	28.51%	23.40%	15.21%	18.11%

external storage may contain sensitive data. For example, “log” is a hot word for both internal and external storage, and logs should be stored on internal storage. However, many developers store the crash log of apps in external storage. The crash log records detailed information about the crashes or abnormal situations that occur during the runtime of apps. By analyzing crash logs, attackers may discover security vulnerabilities in apps and exploit them for attacks, such as code injection and denial-of-service attacks. However, many developers believe that log information is not critical and store it on external storage. Furthermore, the external storage also leaks the following sensitive data: Database, Contact, Signature, QR Code, Screenshot and so on.

Leakage in app logs In this part, we adopted MobiLogLeak (Zhou et al. 2020) to analyze the sensitive data leaked through app logs. MobiLogLeak is a taint analysis-based tool that classifies sensitive information into four categories: network, account, location, and database. For example, the app OSRSHelper¹ leaks the account and username through the following log:

```
DBController: getAccountByUsername:
    account= com.infonuascap.eosrshelper.models.Account @b95a6be
    username=Alice
```

Also, the path-sensitive data flow analysis often leads to path explosion problem, causing overhead. Considering our server’s performance and running time, we only performed log leak analysis for apps less than 10 MB. There are 49,387 apps left in our dataset, which is still considerable and can reflect general statistical characteristics. In addition, we improved MobiLogLeak with the support of multi-process analysis for large-scale measurements.

Based on our statistics, 2.25% of apps leaked sensitive data in their logs, for a total of 5,201 leaks. It is worth noting that our measurements are only a lower bound on actual leakage. When considering large-size apps, the leak may be more severe since they integrate more functions and add more logs for debugging. The usage times and ratios of different log levels for leaking sensitive data are listed in Table 2. It shows that most developers prefer to use the Info and Debug levels to output logs. In addition, many leakages appear in the Warn and Error levels, which is also an alarming development practice. Warn is mainly to remind the developers that there may be potential risks, and Error is primarily used to print the error information of the app. Leaking sensitive data in logs is mainly for debugging purposes, so Verbose, Debug, and Info levels should be used instead of Warn and Error levels. In our measurement, there are mainly two types of data leakage – network and database, of which proportions are 28.44% and 71.56%, respectively. Most network-related leaks are primarily hardware identifiers. The database is mainly the leakage of field information in database tables.

The possible reason for sensitive data leakage is that developers want to confirm that the return value of the function or the value of the field is the expected result. For example, the

¹ Package name: com.infonuascap.eosrshelper

LocLite² app retrieves user tokens by querying the database, and then uses log to print the tokens to determine the success of the database query, resulting in the leakage of user tokens. The leakage of user tokens can lead to a series of security risks, such as unauthorized access, identity theft, and data leakage. The secondary reason is that developers may debug multiple times and not delete log information immediately after each debug. In addition, many developers do not know how to automatically remove logs in apps (Stack-Overflow 2010a). There may be thousands of source files for the current Android app project, and only manually deleting the log will cause much time overhead. Therefore, due to insufficient security awareness, developers do not remove the logs in apps.

Observation #1: *App developers may not distinguish sensitive data well, and such data appears on external storage. In addition, though the leakage in app logs is not very common (< 3%), the leaked data is vital, usually relating to hardware identifiers and databases.*

Guideline 2: Check the validity of file paths when reading files.

In this guideline, we mainly focus on whether developers have checked the validity of file paths when reading files, and the file path should not contain relative paths.

As mentioned in Section 2.2, when developers decompress files or use the `openFile` method for file sharing, they need to check file path. In our dataset, around 10.31% of apps decompress files with 12,958 decompression operations. Among them, 81.24% of operations are vulnerable, and most developers did not sterilize the file paths. Decompressing a zip compressed package has a relatively standard process: (1) Convert the compressed package to an input stream, and further convert it to a `ZipInputStream`. (2) Traverse each file in the compressed package and specify the decompression location of the file. (3) Convert each file to an output stream and extract it to the destination directory. This process may be a bit complicated for inexperienced developers, and the Android official documentation does not provide a concrete example to guide developers.

On another aspect, only 25.51% of apps override the `openFile` method under `ContentProvider` class for file sharing. Such a low ratio is a good phenomenon because if the `openFile` method is overridden in apps without filtering the file path, the apps will be exposed to the risk of path traversal vulnerabilities. The reason developers rarely override the `openFile` method is probably because it is more complicated to implement file sharing through the `openFile` method. A total of 30,134 `openFile` methods are overridden in 27,573 apps, of which only 1.02% (307) sterilize file paths.

In the above two path traversal vulnerabilities, only a small number of developers actively sterilize file paths. The possible reason may be the misleading of incorrect online code examples. We find that the Android official documentation does not provide concrete code examples to check file path validity. Facing the lack of official guidance, an empirical survey by Zhang et al. (2018) shows that developers often learn the use of new APIs through online Q&A forums such as Stack Overflow and often suffer from API misuse issues. To investigate this issue, we reviewed the top answers on Stack Overflow about file unzipping and file sharing using `openFile` method (Stack-Overflow 2010b, 2012b). The result shows that none answers sterilize the file paths, and developers may directly reference these incorrect code snippets into their apps to quickly fulfill the business requirements of apps.

² Package name: com.familylocator.gpstracker

Observation #2: *Only a small fraction of developers check the validity of the file paths when reading files, and Google should provide concrete API usage examples for reference to avoid path traversal vulnerabilities.*

Guideline 3: Apply network security measures.

In this guideline, we mainly focus on whether developers define the NSC files and whether cleartext network traffic is enabled in apps. We determine whether apps define the NSC file by checking the `networkSecurityConfig` attribute of the Application component. If apps define NSC files, we perform XML parsing to obtain specific attributes, such as domain names and cleartext network traffic settings.

NSC file Since the NSC file was added in API level 24, we only analyze the adoption rate of the NSC file in apps with `targetSdkVersion` ≥ 24 . In our dataset, 85,812 apps with `targetSdkVersion` ≥ 24 , and 22.84% of apps have defined the NSC file. Furthermore, the adoption rate of the NSC file for apps with different `targetSdkVersion` is shown in Table 3. The results indicate that the uptake of NSC files among developers is remarkably low for apps `targetSdkVersion` 24 or 25, with an adoption rate falling below 1%. In contrast, there is a marked increase in the adoption of NSC files from `targetSdkVersion` 27 to 28, the rate surged by 15.75%. This surge is likely influenced by Google's decision to enforce a default prohibition on cleartext network traffic starting from API level 28, prompting some developers to use NSC files to deliberately enable cleartext network traffic. When apps with the `targetSdkVersion` ≥ 30 , the adoption rate of NSC file starts to stabilize at approximately 30%. Despite a gradual increase in the adoption of NSC files, the overall rate remains relatively low.

Cleartext network traffic Starting from API level 28, cleartext network traffic support is disabled by default. Our analysis focuses on determining whether apps intentionally enable this feature. Among 70,720 apps with `targetSdkVersion` ≥ 28 , 39.22% enable global cleartext network traffic via the `usesCleartextTraffic` attribute in the Application component, whereas 16.17% utilize the NSC file for this purpose. Developers show a preference for the `usesCleartextTraffic` attribute over the NSC file when enabling cleartext network traffic, likely due to the attribute's simpler implementation compared to configuring the NSC file. Interestingly, 7.72% of apps enable global cleartext network traffic using both methods simultaneously, creating an unnecessary configuration redundancy. Given that from

Table 3 The adoption rate of NSC file by `targetSdkVersion`

TargetSdkVersion	NSC Apps	Total Apps	Percentage
24	5	1,310	0.38%
25	21	2,232	0.94%
26	207	5,992	3.45%
27	161	5,558	2.90%
28	2,638	14,146	18.65%
29	4,046	17,323	23.36%
30	6,243	20,298	30.76%
31	3,709	10,597	35%
32	1,013	3,451	29.35%
33	1,558	4,905	31.76%

API level 24 onwards, the system will disregard the `usesCleartextTraffic` attribute if an NSC file is defined, this redundancy can lead to confusion among app maintainers regarding the actual configuration of the app's network security policies, thereby adding unnecessary complexity to the app's security setup.

Only 12.45% of apps enable cleartext network traffic for specific domains without enabling global cleartext network traffic. This trend indicates that developers tend to prefer enabling global cleartext network traffic directly, rather than selectively enabling cleartext network traffic support for specific domains as needed. Among the 8,806 apps, cleartext network traffic has been enabled for 24,950 domains. Notably, 23.22% of these domains are local loopback addresses. Moreover, 4,498 of these domains are represented by IP addresses, with a significant 78.39% of these IPs classified as LAN (Local Area Network) addresses. Furthermore, we performed an analysis comparing package names with domain names to determine potential ownership by app developers. To enhance accuracy, we excluded common keywords such as `com` and `cn` from this comparison. Our analysis revealed matching keywords in 2,944 instances of domain and package names, suggesting that these domains accessed within the apps are likely owned by the respective app developers. For the local loopback address, developers may enable cleartext network traffic for debugging purposes. Even if developers do not disable cleartext network traffic for the local loopback address after apps are released, it should not pose any risks to apps. However, enabling cleartext network traffic for LAN addresses or the developer's own websites can still expose apps to the risk of MITM attacks.

We conducted tests on 15,631 domain names, excluding local loopback and LAN addresses. 16.22% of these domain names are subject to security downgrades, meaning they support the HTTPS protocol, yet developers continue to access them using the HTTP protocol within apps. It may be because these domains do not support the HTTPS protocol when apps are published. However, these websites have recently started supporting HTTPS, but developers do not update the NSC files of apps on time, leading to security downgrades. Furthermore, 1,247 domain name developers have actively disabled cleartext network traffic, of which 191 domain names contain the keyword "api". Developers typically engage in data transmission and reception when using APIs provided by third-party services. As a result, they often disable cleartext network traffic to prevent data leaks.

Observation #3: *Since API level 30, the adoption rate of the NSC file remained around 30%. Furthermore, many developers enable global cleartext network traffic directly within apps starting from API level 28.*

Guideline 4: Exported components should be protected by appropriate custom permissions.

In this guideline, we mainly focus on the exportation of app components and the usage of custom permissions for protecting exported components. We mainly analyzed apps' `AndroidManifest.xml` files, and parsed all the four components defined and exported, as well as the custom permissions defined and unused.

Exported components Among the 108,091 apps, 74.14% of apps have at least one exported Activity (the default export of `MainActivity` has been excluded). For Service, Receiver, and Provider, the percentages are 57.93%, 69.90%, and 10.10%, respectively. Activity exports are common in apps because developers need to use third-party SDKs to receive data returned from third-party apps. To receive the returned results correctly, third-party apps must be able to access the Activity in the developer's app to pass data, so the Activity must be exported.

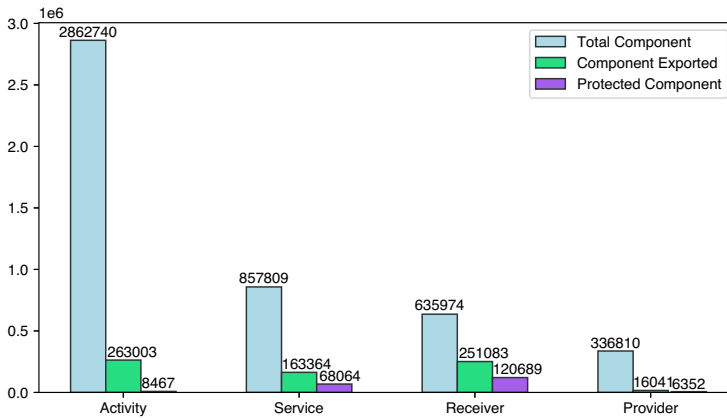


Fig. 3 Statistics of major components used in apps

For example, 19,943 apps have incorporated the Facebook SDK, enabling users to log in with their Facebook accounts without the need to register a new account. If the user has installed the Facebook app and login information should be passed from the Facebook app, setting the exported attribute value to false will make the Activity private and unable to receive data.

Also, as plotted in Fig. 3, the total export percentages of four major components in apps are 9.19%, 19.04%, 39.48%, and 4.76%. The export rate of Activity and Provider is relatively low. For Activity, it is likely due to the default attribute configurations of Android Studio. Android Studio will set them exportable by default when developers create the Service, Receiver, and Provider components. However, to Activity, the default attribute is unexportable. Furthermore, unlike Service and Receiver, Provider is used to share data between apps, and developers typically do not proactively share the data collected by their own apps with other apps. Exported components can lead to the risks of information leakage, phishing, and denial-of-service (MITRE 2013). For example, suppose the exported Activity does not handle exceptions to the data obtained by `Intent.getXXXExtra()`. In that case, an attacker can crash the app by sending empty, abnormal, or malformed data.

Protected components On the other hand, only 3.22% of exported Activity are protected by custom permissions. The percentages of Service, Receiver, and Provider being protected are higher than Activity, say 41.66%, 48.07%, and 39.6% respectively. The reason is that, after using the third-party SDK, the Activity that receives the callback result usually does not set any permissions to ensure this Activity's accessibility.

We extracted the protection levels of custom permissions used to protect the four major components, as shown in Fig. 4. As suggested by Guideline 4, the protection level should be signature or signatureOrSystem³. According to our statistics, most configurations are correct, say Activity: 97.98%, Service: 99.86%, Receiver: 98.66%, and Provider: 96.39%. A few developers use normal-level permissions to protect components, which is insecure. A normal-level permission can be obtained by any app. The reason is mainly due to the default value of the `protectionLevel` attribute of the `<permission>` element. If developers do not explicitly specify the protection levels of defined custom permissions, the default level

³ In most cases, signature and signatureOrSystem are equivalent.

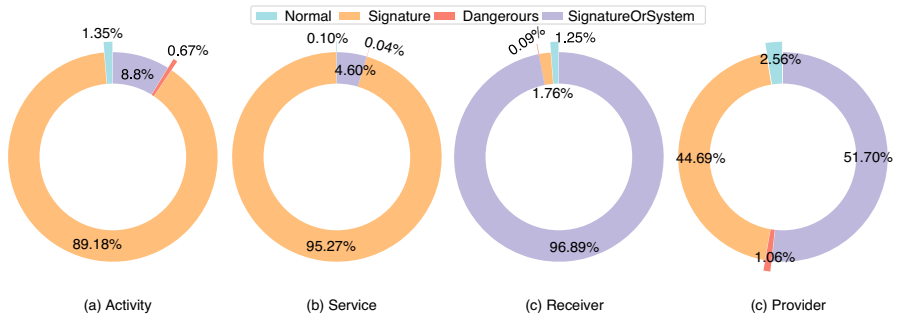


Fig. 4 Protection levels of custom permissions

will be assigned as normal. Developers may believe that the default protection level is secure to protect components. Also, some developers use dangerous-level custom permissions to protect components, which is also not recommended by Google. Creating dangerous-level permissions will introduce user decisions, and users may not have enough security knowledge to make appropriate decisions.

For ContentProvider, Google has implemented fine-grained permission control, dividing it into read-only or write-only permissions. 79.61% (5,057) usage cases apply read-only permission to ContentProvider, and 1.21% (77) usage cases apply write-only permission. However, 19.18% (1,218) usage cases still do not implement this fine-grained protection, and apps are granted both read and write permissions, although most apps may only need write or read permissions.

Custom permission usage For custom permissions, 1.07% of apps have the over-privilege issue. That is, custom permission is defined without use. 78.7% of unused custom permissions have the same name as the system permissions. This could be due to developers mistakenly using the `<permission>` tag instead of the `<use-permission>` tag when requesting system permissions. Another portion of unused custom permissions may be because these custom permissions are used to protect components in the previous versions of apps. However, these components have been removed in the recent versions, and developers did not remove the corresponding custom permissions, as we explained in the conference version of this work.

Observation #4: *The app component export operations are widespread. Only a small part of the exported components are protected by custom permissions. Among these use cases, most developers can set the protection levels of custom permissions correctly.*

Guideline 5: Disable JavaScript support and local file loading when using WebView objects.

In this guideline, we mainly focus on whether apps enable JavaScript interface when using the WebView objects and ensure that the loaded content is trustworthy. Also, we checked whether Webview objects enabled settings related to local file loading.

JavaScript interface To determine whether apps enable JavaScript interface when using the WebView objects, we primarily check whether apps invoke the `setJavaScriptEnabled` method and set its parameter to true. If apps enable the JavaScript interface, we will further check whether apps override the `shouldOverrideUrlLoading` callback method to to

ensure that the content loaded by WebView objects is trustworthy. Among 108,091 apps, 54.3% have enabled JavaScript interfaces while using WebView objects, totaling 186,191 usage cases. When JavaScript interface is enabled, the Webview object typically has three ways of handling the content about to be loaded:

- 11.48% of usage cases do not check the content to be loaded. Specifically, they do not override the `shouldOverrideUrlLoading` callback method. This part of developers only focuses on implementing the functionality of apps without paying attention to the risks associated with loading content in WebView objects.
- Although 56.87% of usage cases override the `shouldOverrideUrlLoading` callback method, they do not check the content that the WebView objects are about to load. This occurs because the default behavior when clicking on links within WebView objects is to automatically open them in the external browser. To alter this, developers override the `shouldOverrideUrlLoading` callback method. However, the primary intention behind this override is not to scrutinize the content being loaded. Instead, it is to ensure that links clicked within the WebView objects open internally within the WebView itself, rather than defaulting to the external browser. This approach is exemplified in the code snippet referred to as Listing 1. However, Google explicitly discourages using this approach in Android development documentation (Google 2024n). This approach will cancel the current load and start a new load with the same URL. The correct way to continue loading a given URL is to simply return false, without calling `WebView.loadUrl`.

```

1 @Override
2 public boolean shouldOverrideUrlLoading(WebView view,
3     String url) {
4     view.loadUrl(url);
5     return true;
6 }

```

Listing 1 Example of `shouldOverrideUrlLoading`.

Typically, the function returning true usually indicates the smooth execution of the program, while returning false signifies that the function is unable to complete its intended task successfully. Consequently, developers are inclined to use return true instead of return false with the belief that this will ensure WebView objects successfully load the content.

- 31.64% of usage cases override the `shouldOverrideUrlLoading` callback method and check the content about to be loaded. Although these cases involve content checking, many fail to adopt the allowlist approach recommended by Google. They only conduct basic checks on the URI to be loaded, such as preventing the loading of URLs that use the HTTP protocol.

Many websites rely on JavaScript to dynamically generate and update content. Enabling JavaScript support allows the WebView object to correctly display and interact with this dynamic content. To provide a better user experience, developers may enable JavaScript support in WebView objects. However, they overlooked the importance of verifying the trustworthiness of the loaded content.

Local file loading Local file loading primarily focuses on whether developers use the `setAllowFileAccess`, `setAllowFileAccessFromFileURLs`, and `setAllowUniversalAccessFromFileURLs` APIs when using WebView objects to

Table 4 Usage of file protocol

API Name	Enabled	Disabled
setAllowFileAccess (API $\leq$ 29)	93.60%	6.40%
setAllowFileAccess (API $>$ 29)	0	100%
setAllowFileAccessFromFileURLs	1.80%	98.20%
setAllowUniversalAccessFromFileURLs	2.04%	97.96%

*: In API level $\leq$ 29, setAllowFileAccess defaults to enabled, while in API level $>$ 29, it defaults to disabled. setAllowFileAccessFromFileURLs and setAllowUniversalAccessFromFileURLs are always disabled by default

enable file access, access files other than the currently loaded file, and enable cross-origin access. To determine whether apps have configured settings related to local file loading, we primarily check if apps call the aforementioned APIs and set their parameters to true. Table 4 illustrates the enabling patterns of settings related to local file loading. The default values of APIs play a crucial role in determining whether apps enable or disable these settings. When the default value is set to true, only a small number of developers take the initiative to manually disable settings associated with local file loading. Conversely, in cases where the default value is false, just a minor portion of developers actively enable these settings.

Setting setAllowFileAccess to true is a prerequisite for enabling local file access. Since loading local files typically involves nested references to other files, it is essential to set setAllowFileAccessFromFileURLs to true. For example, HTML files generally contain references to JavaScript and CSS files. Additionally, some local files may load network resources, such as advertisements from remote servers. Without setting setAllowUniversalAccessFromFileURLs to true, these network requests might be blocked, causing the ads to fail to load. If developers need to use Webview objects to load local files, they should carefully review the content loaded by WebView objects. Some developers load local files from insecure locations. For example, the FishingNemo⁴ app uses WebView objects to load content passed through intent. However, the values carried by the intent could come from any app, making it vulnerable to injection attacks.

Observation #5: *Approximately 50% of apps enable the JavaScript interface when using WebView objects, but rarely use the allowlist mechanism to check the content about to be loaded. Additionally, the enabling of the file protocol is greatly affected by the default value.*

Guideline 6: Avoid introducing intent vulnerabilities.

In this guideline, we focus primarily on three types of intent vulnerabilities (implicit intent hijacking, intent scheme hijacking, and intent redirection) within apps.

Implicit intent hijacking We mainly identify explicit and implicit intents by checking whether apps explicitly specify the target component of intents when dispatching intents. For instance, in the case of explicit intents, apps typically employ methods like `Intent.setComponent` or `Intent.setClass` to specify the intent's target component. Among 108,091 apps, 91.12% of apps use implicit intents. Furthermore, our analysis revealed 7,026,438 instances of intent dispatch, among which 62.67% are explicit intents, and 37.33% are implicit intents.

⁴ Package name: com.wFishingNemo_9006313

If apps use implicit intent, the following two methods can be employed to mitigate the risks caused by implicit intent: (1) Using the `Intent.setPackage` method to explicitly specify the package name of target component of intents to prevent malicious apps from receiving intents. Only 18.39% of instances of intent dispatch employ this strategy. (2) Use the app chooser to specify the list of target apps of intents through `Intent.createChooser` method. This app chooser not only lists potential target apps but can also present a descriptive title, guiding users to select the appropriate target app. However, the adoption rate of these two strategies is very low when dispatching implicit intents, especially the app chooser.

- Only 0.66% of instances use the app chooser and restrict the target apps of intents.
- 8.42% of instances, although using the app chooser, do not restrict the list of target apps of intents. If an app uses the app chooser without restricting the target apps, the system will by default display all apps capable of receiving the intent. The default implicit intent often comes with an option always to open it with a specific app. If the user accidentally clicks this option, they cannot change the target apps of intents. Therefore, by using the app chooser, apps can select target apps each time when dispatching intents, preventing persistent effects caused by users' accidental operations.
- 90.92% of instances of intent dispatch do not employ the app chooser strategy.

Developers use implicit intents primarily because they offer greater flexibility compared to explicit intents. This approach allows users greater choice, as exemplified when opening a PDF attachment from an email. In such cases, an implicit intent enables users to select their preferred PDF viewer. However, this flexibility also introduces the risk of the PDF file being hijacked by malicious apps.

```

1 Intent intent = Intent.parseUri(url);
2 intent.addCategory("android.intent.category.BROWSABLE");
3 intent.setComponent(null);
4 intent.setSelector(null);

```

Listing 2 Example of restricting intent to be dispatched only to component with BROWSABLE Intent filters.

Intent scheme hijacking and redirection To detect intent scheme hijacking vulnerabilities, our primary goal is to identify whether apps use the `Intent.parseUri` method to parse Uris. To detect intent redirection vulnerabilities, we mainly identify whether apps will obtain nested intents through the related method, such as `Intent.getParcelableExtra`, and dispatch them.

Among 108,091 apps, 11.33% of apps use `Intent.parseUri` to generate intents, totaling 30,655 usage cases. For intents generated by `Intent.parseUri`, developers can restrict that these intents are dispatched exclusively to components with BROWSABLE intent filters, preventing them from being sent to other private components, as shown in Listing 2. 15.69% of usage cases only enable one or two of the three constraint steps, while only 3.12% of usage cases enable three constraint steps. If apps do not fully implement the three constraint steps mentioned above, they may still be vulnerable to intent scheme hijacking. On the other hand, 1.12% of apps contain intent redirection, totaling 1,215 usage cases. Apps can mitigate the risk of intent redirection vulnerability by verifying the target component of redirected intents to ensure it is a secure component. Among 16,919 usage cases, developers checked the target components of redirected intent only once. This means that almost no developers implement any form of restrictions on redirected intent.

Apps face the risk of Intent Scheme Hijacking mainly because `Intent.parseUri` allows dynamically generating intent objects from URI strings, enhancing flexibility in handling dynamic content. Additionally, using URI strings simplifies code and ensures compatibility and interoperability between apps. Finally, it simplifies the persistence and recovery of intent, especially in complex navigation logic or state recovery scenarios. The primary cause of intent redirection vulnerabilities is that apps do not verify the trustworthiness of the source when receiving intents from third-party apps, or do not check the components to which they are redirected, thereby failing to prevent redirection to the app's private components.

Observation #6: *Implicit intent hijacking has a higher prevalence in apps compared to intent scheme hijacking and intent redirection, accounting for approximately 90%. Additionally, apps rarely follow Google's recommendations to impose restrictions on three intent vulnerabilities.*

Guideline 7: Use secure modes when creating files.

In this guideline, we mainly focus on the file mode developers use when creating files. In detection, we extracted the integer value corresponding to the file mode by identifying the API that creates `SharedPreferences`, database, and private files. After that, the extracted values will be converted to file modes for further analysis.

Usage of secure file modes We counted the usage times of different file modes for creating `SharedPreferences`, database, and private files in 108,091 apps, as shown in Table 5. The corresponding percentages of creating files in secure modes are 98.68% (2,656,455), 80.54% (11,209), and 96.72% (107,751), respectively.

Usage of insecure file modes There are mainly three risky file mode usage cases:

- *Incorrect usage of a single file mode.* The incorrect usage of a single mode indicates that the developers use the mode marked insecure in Table 5. The proportions of such errors used in `SharedPreferences`, database, and private files are 1.24% (33,335), 1.04% (145), and 2.73% (3,044), respectively.
- *Incorrect usage of file mode combinations.* Android allows the assigned file mode can be a combination of multiple modes. It will be more harmful if the combination includes multiple insecure modes. For example, `[MODE_WORLD_READABLE + MODE_WORLD_WRITEABLE]` can grant other apps read and write permissions to the file. In our measurement, the proportions of insecure combinations are 0.03% (834), 0.01% (2), and 0.51% (564), respectively.
- *Usage of unsupported file modes.* Some developers used unsupported modes to create files. For example, `SharedPreferences` files only support four modes – `MODE_PRIVATE`, `MODE_WORLD_READABLE`, `MODE_WORLD_WRITEABLE`, and `MODE_MULTI_PROCESS`. However, some developers specify other modes such as `MODE_APPEND` or some unknown types of integer values. That is, these developers may be confused about the modes supported by each type of file. Also, some developers even do not know that the file mode is a fixed constant value, and they fill in an integer value casually. The proportions of this error used in the above three kinds of files are 0.05% (1,436), 18.41% (2,562), and 0.04% (47).

Table 5 Configured file modes when creating files

Mode	Value	Security	SP§	DB§	PF§
MODE_PRIVATE	0x0000	Secure	2,656,455	11,196	103,442
MODE_WORLD_READABLE	0x0001	Insecure	7,687	135	2,757
MODE_WORLD_WRITEABLE	0x0002	Insecure	1,024	10	287
MODE_MULTI_PROCESS	0x0004	Insecure	24,624	N/A*	N/A
MODE_ENABLE_WRITE_AHEAD_LOGGING	0x0008	Secure	N/A	4	N/A
MODE_NO_LOCALIZED_COLLATORS	0x0010	Secure	N/A	9	N/A
MODE_APPEND	0x8000	Secure	N/A	N/A	4,309
MODE_INSECURE_COMBINATION†	Multiple◇	Insecure	834	2	564
MODE_ERROR_USE‡	Multiple	Insecure	1,436	2,562	47

§: SP represents SharedPreferences, DB for database, PF for private file.

*: N/A represents the file does not support this mode.

†: MODE_INSECURE_COMBINATION denotes insecure file creation mode combinations.

‡: MODE_ERROR_USE denotes a wrong mode is used to create the file.

◇: Multiple denotes multiple values in this mode.

When developers create database files through the `openOrCreateDatabase` method, there's a higher chance they might use unsupported file modes. Among the 2,562 database files created using unsupported modes, 2,548 are created with the file mode set to `CREATE_IF_NECESSARY`. However, the `openOrCreateDatabase` method does not support the `CREATE_IF_NECESSARY` mode. This mode applies to the `openDatabase` method. After testing, the files created with unsupported modes will be treated as `MODE_PRIVATE`. However, there are no security guarantees, i.e., with later versions of Android, file permissions created with unsupported modes may become dangerous, such as becoming world readable and writable.

The possible reason why developers use insecure file mode is to achieve file sharing between apps. If the file is not set to `MODE_WORLD_READABLE`, the shared apps cannot obtain the shared content. For example, the app `AerialPlatformsWallpaper`⁵ shares the file to the mail app as an attachment of the mail, as shown in Listing 3. In this case, `AerialPlatformsWallpaper` app specifies that the file is stored in the internal storage and sets the file mode to `MODE_WORLD_READABLE` through the `openFileOutput` method (line 5). Then, it will convert the file to an insecure file: `Uri` via the `fromFile` method (line 7). File `Uri` access control needs to open the underlying file system permissions, and the open permissions will be available to any apps until the next close.

```

1 private void addTextAttachment(String str, String str2) {
2   FileOutputStream fos = null;
3   ...
4   try {
5     fos = this.mContext.openFileOutput(str, 1);
6     fos.write(str2.getBytes());
7     this.mEmailAttachments.add(Uri.fromFile(new File(this.
        mContext.getFilesDir() + File.separator + str)));
    ...}}

```

Listing 3 Example of file sharing.

Observation #7: *Most developers create files using secure modes, but some use insecure file modes to implement the file-sharing function.*

Guideline 8: Avoid using non-resettable hardware IDs.

In this guideline, we mainly focus on whether developers use non-resettable hardware identifiers, such as IMEI, S/N, MAC Address, and SSAID. We mainly analyzed the usage times of hardware identifiers in apps by parsing all the APIs to obtain hardware identifiers. Then, we also got app's `targetSdkVersion` to analyze the average usage of hardware identifiers in different `targetSdkVersion`.

Overall statistics In our app dataset, 63.3% of apps used at least one hardware identifier. We counted the total number of the four hardware identifiers, as listed in Table 6. We find that developers are more inclined to use IMEI and SSAID compared to S/N and MAC Address. It is mainly because IMEI exists in every version of Android and has been added since API level 1. In addition, it is more stable and easier to obtain than other identifiers. Similarly, SSAID can be obtained through the `Settings.Secure.ANDROID_ID` constant without any restrictions starting from API level 26 (in our dataset, 76.11% of apps with `targetSdkVersion` $\geq$ 26). Unlike IMEI and SSAID, there are some restrictions for obtaining MAC Address and S/N. From API level 23, if apps do not have the

⁵ Package name: `com.petrguskov.AerialPlatformsWallpaper`

Table 6 Total usage of hardware identifiers

Identifier	IMEI	S/N	MAC Address	SSAID
Amount	112,828	36,164	30,131	195,961

LOCAL_MAC_ADDRESS and ACCESS_FINE_LOCATION permissions, calling the SDK API to obtain the MAC address will return a fixed string value. However, many developers still obtain MAC Address through some Google-discouraged methods, such as scanning various network interfaces to retrieve MAC Address. Although S/N is relatively stable and not easily changed, it is used far less than IMEI and SSAID. The possible reason is that the Build.Serial API is not available in all Android phones, and it was added at API level 9. Also, the value returned by Build.Serial may be inconsistent with the value listed in the Settings of the mobile phone (Overflow 2015; Stack-Overflow 2012a). That is to say, the actual S/N cannot be acquired through the Build.Serial API. Therefore, many developers rely on Java reflection to access non-SDK APIs for reading the Android.os.SystemProperties configuration file to obtain the actual S/N. Non-SDK APIs have no compatibility guarantee and may be removed from the Android framework at any time, causing runtime crashes in apps (Yang et al. 2022).

```

1 public void UploadMedia(String str, boolean z, final
    OnCompleteListener listener) {
2   InitAWS(mContext);
3   ...
4   mUserId = getDeviceId(mContext);
5   ...
6   StringBuilder sb = new StringBuilder();
7   ...
8   sb.append(mUserId.toUpperCase());
9   sb.append("-");
10  sb.append(System.currentTimeMillis());
11  sb.append(".png");
12  final String sb2 = sb.toString();
13  ...
14  PutObjectRequest pbr = new PutObjectRequest("skinseed",
    sb2, new File(str));
15  ...
16  Upload upload = new TransferManager(this.
    credentialsProvider).upload(pbr); ...}

```

Listing 4 Example of device binding using IMEI.

App developers mainly use hardware identifiers to bind the app to the device (user) for unique identification. For example, Skinseed⁶ app uses AWS (Amazon Web Services) to store and manage images. When uploading images to Amazon's cloud storage server (line 16), it is necessary to use the device's IMEI (line 4) as the device's identifier to determine which device uploaded the images, as shown in Listing 4. However, since the hardware identifier is usually associated with the device user, it may harm the user's privacy. Device tracking can be performed when a malicious attacker obtains the victim's hardware identifier.

Usage trend From API level 29, Google has imposed stricter restrictions on retrieving IMEI and S/N. If the apps with `targetSdkVersion ≥ 29`, obtaining some hardware identifiers

⁶ Package name: com.africasunrise.skinseed

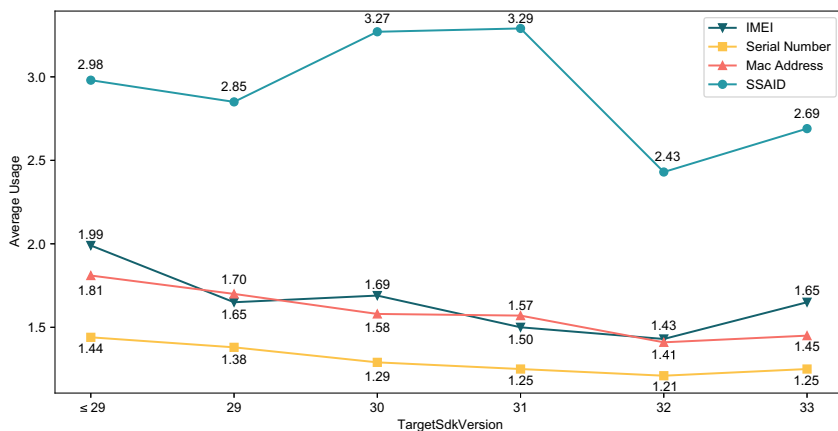


Fig. 5 Used hardware identifiers by targetSdkVersion

will involve a compatibility problem. For example, third-party apps cannot obtain IMEI and S/N. If they are queried in code, a `SecurityException` will be thrown. The average reference count of the four hardware identifiers in apps is shown in Fig. 5. Due to Google's restrictions on accessing IMEI, S/N, and MAC Address, the average reference count of these hardware identifiers has shown a downward trend overall in API level 29 and later. On the contrary, due to the lack of restrictions on accessing SSAID, developers increased the use of SSAID.

Observation #8: *Using non-resettable hardware IDs may lead to the risk of device tracking. In practice, many app developers still use hardware identifiers for the device (user) binding and identification. This problem is gradually easing with the API restriction of Google after API level 29.*

🛡️ Guideline 9: Improving app code quality to mitigate man-in-the-middle attacks.

In this guideline, our primary focus is whether apps correctly validate hostnames and certificates of server in SSL/TLS communication.

Androbugs (Lin 2015) is an Android app vulnerability analysis framework that can help developers find potential security vulnerabilities in apps. Therefore, we primarily use Androbugs to evaluate whether apps validate hostname and certificate of server during network communication. The verification situation of the hostnames and certificate of server in apps is shown in Table 7. The proportions of the four types of vulnerabilities in apps are 11.88%, 4.09%, 19.27%, and 9.69%, respectively. The presence of any of the vulnerabili-

Table 7 The verification situation of the hostname and certificate of server in apps

Vulnerability Name		Apps	Total	Average
WebView SSL Error		12,841	16,196	1.26
Unsafe Hostname Verification	Allow All Hostnames	4,418	5,221	1.18
	Insecure Verification	20,831	25,654	1.23
Unsafe X.509 TrustManager	10,474	18,392	1.76	

ties in the above table can expose apps to the risk of MITM attacks. Among 108,091 apps, 32.36% of apps have at least one of the above vulnerabilities. In addition, among the apps with the four vulnerabilities mentioned above, the average number of occurrences for each vulnerability is less than 2 times. Developers are more inclined to use the `verify` method of the `HostnameVerifier` interface to accept all hostnames, rather than utilizing the `ALLOW_ALL_HOSTNAME_VERIFIER` field. They often provide an empty implementation for the `verify` method.

Ignoring hostname and certificate of server verification may be because developers may want to access websites that use self-signed certificates, which can be either their own websites or third-party websites. Since self-signed certificates do not form a trust chain with the certificates built into the Android OS, apps may throw certificate verification failures when validating the certificate, resulting in the inability to access the target website. Therefore, developers often choose not to validate the hostname and certificate of server to access self-signed websites. Moreover, the server's hostname and certificate verification strategy can be configured within NSC files. As outlined in Guideline 3, the low adoption rate of NSC files may also contribute to the vulnerability of apps to MITM attacks.

Observation #9: *About 33% of apps are at risk of MITM attacks. Developers may ignore hostname and certificate of server verification to access self-signed websites.*

5 Discussions

5.1 Cross-category differences

The uneven distribution of the number of apps in each category on Google Play has resulted in a similarly uneven distribution in our dataset, as shown in Section 3.3. Therefore, we selected the five categories with the largest number of apps (GAME, EDUCATION, BUSINESS, TOOLS, FINANCE) as representatives to explore whether there are differences in guideline violations among apps from different categories, as shown in Table 8. The violation of Guidelines 1 - 9 by apps in the five categories did not show significant differences in most cases. However, apps in the EDUCATION category have a higher probability of violating

Table 8 The violation rates of guidelines among apps from different categories

	GAME	EDUCATION	BUSINESS	TOOLS	FINANCE
Guideline 1	0.63%	1.57%	2.0%	1.56%	1.40%
Guideline 2	47.37%	50.32%	23.89%	18.92%	31.33%
Guideline 3	83.12%	88.79%	84.34%	84.02%	79.10%
Guideline 4	85.93%	81.45%	77.25%	70.87%	78.86%
Guideline 5	50.41%	66.59%	45.15%	38.43%	51.82%
Guideline 6	79.66%	88.32%	85.29%	83.54%	84.18%
Guideline 7	7.28%	37.04%	12.97%	12.22%	13.5%
Guideline 8	64.86%	68.44%	66.61%	50.87%	69.79%
Guideline 9	28.15%	50.36%	38.57%	23.53%	40.91%

Guidelines 1 - 9. Particularly for Guideline 7, the violation rates for the other four categories are around 10%, while the violation rate for apps in the EDUCATION category reaches 37.04%. By investigating the code call locations that violate Guideline 7, it is primarily found that these EDUCATION category apps integrate several third-party libraries, such as Razorpay SDK, Zoom SDK and Facebook SDK. These third-party libraries share configuration information using insecure file modes across multiple apps.

5.2 Lessons learned

The reasons for the low adoption rates of Guidelines 2, 3, 4, 5, 6, 8, and 9 are different. Developers are committed to rapidly complete the development and release of apps to compete for market share, a practice that often conflicts with the security requirements of the apps. Developers will likely continue to make these mistakes unless they receive adequate education on these security issues. To increase the adoption rate of these guidelines, we suggest starting with Google and app market maintainers. They can adopt the following suggestions:

- It would be beneficial for Google to provide developers with concrete code examples that demonstrate the implementation of complex development operations. By offering practical examples, developers can better understand how to apply the guidelines in their apps smoothly. In addition, adopting a security-by-default design approach for Android Studio can significantly contribute to the improved adoption rate of the guidelines. This means implementing default security measures within the development environment itself, such as enforcing secure coding practices and providing built-in security features.
- App marketplace maintainers should design the corresponding automated detection tools based on the security guidelines provided by Google and integrate them into the app marketplace's review process to prevent the release of apps that violate the security guidelines.

5.3 Threats to validity

Static code analysis. In our analysis, we decompiled apps mainly using AndroGuard, a Python-based static analysis tool. Since static analysis does not execute code, some identified behaviors (compliance and violation of guidelines) may not be triggered in apps at runtime. On the other hand, dynamic analysis is unsuitable for large-scale measurements due to its analysis efficiency and code coverage issues.

False negative results. For each guideline compliance and violation, developers may have diverse implementations. Although we have considered most implementation cases, there are inevitably omissions. Even so, we measured 108,091 apps, and the guidelines compliance and violations extracted from such a large number of apps reflect the overall characteristics.

Violation reasons analysis. We utilized various methods to analyze the possible reasons for guideline violations. To validate our analysis and obtain the actual reasons for guideline violations, we need to conduct large-scale app developer studies. As part of this work, we sent out 500 E-mails to app developers for reason confirmations. Unfortunately, we received only 13 responses, so we did not present app developer studies results.

6 Related work

Some previous works focus on the topics of API misuse and developer behaviors. Here we review these works.

API misuse (Egele et al. 2013) evaluated the cryptographic APIs used in Android and found that 10,327 of 11,748 apps had at least one cryptographic misuse based on CryptoLint. Diao et al. (2019) analyzed the Android accessibility APIs framework. They found that the accessibility APIs are widely misused, and developers use the accessibility APIs to bypass the permission restrictions of the Android OS. Shao et al. (2016) proposed SInspector to check for potential misuse of Unix domain sockets in apps and system daemons. A total of 14,644 apps and 60 system daemons were analyzed, and 45 apps and 9 system daemons were found to be vulnerable. Oltrogge et al. (2021) analyzed the use of Network Security Configuration (NSC) in Android, and 88.87% of apps misused NSC to cause security degradation. Luo et al. (2018) proposed the MAD-API framework to detect API misuse problems in apps with the evolution of Android APIs. They found that 93.13% of the evaluated apps had API misuse problems, and the total number of misuses reached 1,241,831. Li et al. (2021) present ARBITRAR, which detects API misuses by interacting with users. They checked the usage of 18 target API methods in 21 C++ programs and discovered 40 bugs. Lyons et al. (2023) systematically studied and analyzed the logging of sensitive data in the Android ecosystem, discovering that both user-installed apps and system components, as well as pre-installed apps, leak sensitive data through logs.

Developer behaviors Li et al. (2021) surveyed the posts on StackOverflow and found the classes that frequently caused usage obstacles but not frequently used, and also revealed some causes of API usage barriers. Zhang et al. (2018) found that developers may learn new API usage from online Q&A sites. They analyzed 217,818 StackOverflow posts using ExampleCheck and found that 31% posts may have potential API usage violations. Li et al. (2020) found that most of the deprecated APIs are accessed by apps through popular libraries, and library developers are more likely than app developers to update deprecated APIs. Vásquez et al. (2013) analyzed the factors affecting the success of Android apps, measuring 7,097 Android apps, and found that heavy use of fault- and change-prone APIs negatively affected the success of these apps. In another direction, Vásquez et al. (2014) studied the developers' responses to API changes. When API behavior is modified, Android developers usually have more questions and triggered more discussions on StackOverflow. Andow et al. (2020) proposed POLICHECK, which combines program analysis and privacy policy analysis to construct a flow-to-policy consistency model that considers data-receiving entities. They found that 42.4% of apps either incorrectly disclosed or failed to disclose their privacy-sensitive data flows.

Unlike the above research, in this paper, from the perspective of Google's app security development guidelines, we evaluate the deployment status of security guidelines. To the best of our knowledge, we are the first to conduct an evaluation of the security guidelines provided by Google.

7 Conclusion

In this paper, we evaluated whether app developers followed the Android official security guidelines and investigated the reasons behind the results. Based on the OWASP MASVS standard, we have selected nine security guidelines, covering the aspects of storage, network communication, platform interaction, and code quality. The high-level result shows that developers did not follow seven of these nine guidelines well. The possible reasons of guideline violations come from both parties, Google and app developers.

Acknowledgements We thank the anonymous reviewers for their insightful comments. This work was supported by Taishan Young Scholar Program of Shandong Province, China (Grant No. tsqn202211001), Shandong Provincial Natural Science Foundation (Grant No. ZR2023MF043), and Xiaomi Young Talents Program.

Data Availability The raw data obtained and analyzed during the current study are available at <https://doi.org/10.5281/zenodo.12602762>.

Declarations

Conflicts of interest The authors declared that they have no conflict of interest.

References

- Allix K, Bissyandé TF, Klein J, Traon YL (2016) Androzoo: collecting millions of android apps for the research community. In: Kim M, Robbes R, Bird C (eds) Proceedings of the 13th international conference on mining software repositories (MSR), Austin, TX, USA, May 14–22, 2016. <https://doi.org/10.1145/2901739.2903508>
- Andow B, Mahmud SY, Whitaker J, Enck W, Reaves B, Singh K, Egelman S (2020) Actions speak louder than words: entity-sensitive privacy policy and data flow analysis with PoliCheck. In: Proceedings of the 29th USENIX security symposium (USENIX-Sec), Boston, MA, USA, August 12–14, 2020. <https://www.usenix.org/conference/usenixsecurity20/presentation/andow>
- C R (2021) Over 60% of android apps have security vulnerabilities. <https://atlasvpn.com/blog/over-60-of-android-apps-have-security-vulnerabilities>
- Diao W, Zhang Y, Zhang L, Li Z, Xu F, Pan X, Liu X, Weng J, Zhang K, Wang X (2019) Kindness is a risky business: on the usage of the accessibility APIs in android. In: Proceedings of the 22nd international symposium on research in attacks, intrusions and defenses (RAID), Beijing, China, September 23–25, 2019. <https://www.usenix.org/conference/raid2019/presentation/diao>
- Egele M, Brumley D, Fratantonio Y, Kruegel C (2013) An empirical study of cryptographic misuse in android applications. In: Proceedings of the 20th ACM SIGSAC conference on computer and communications security (CCS), Berlin, Germany, November 4–8, 2011. <https://doi.org/10.1145/2508859.2516693>
- Gao J, Li L, Kong P, Bissyandé TF, Klein J (2021) Understanding the evolution of android app vulnerabilities. *IEEE Trans Reliab* 70(1):212–230
- Google (2023a) android:exported. <https://developer.android.com/topic/security/risks/android-exported>
- Google (2023b) Fixing a zip path traversal vulnerability. <https://developer.android.com/topic/security/risks/zip-path-traversal>
- Google (2023c) Implicit intent hijacking. <https://developer.android.com/privacy-and-security/risks/implicit-intent-hijacking>
- Google (2023d) Intent redirection. <https://developer.android.com/privacy-and-security/risks/intent-redirection>
- Google (2023e) Path traversal vulnerability. <https://developer.android.com/topic/security/risks/path-traversal>
- Google (2023f) Unsafe HostnameVerifier. <https://developer.android.com/topic/security/risks/unsafe-hostname>
- Google (2023g) Unsafe X.509 TrustManager. <https://developer.android.com/topic/security/risks/unsafe-trustmanager>

- Google (2024a) Add a network security configuration. <https://developer.android.com/privacy-and-security/security-best-practices#network-security-config>
- Google (2024b) Address webview SSL error. <https://support.google.com/faqs/answer/7071387>
- Google (2024c) Android permission protection level. <https://developer.android.com/guide/topics/manifest/permission-element>
- Google (2024d) App security best practices. <https://developer.android.com/privacy-and-security/security-best-practices>
- Google (2024e) Application sandbox. <https://source.android.com/security/app-sandbox>
- Google (2024f) Apply signature-based permissions. <https://developer.android.com/privacy-and-security/security-best-practices#apply-signature-based-permissions>
- Google (2024g) Intent scheme hijacking. <https://support.google.com/faqs/answer/9101196>
- Google (2024h) Mitigate security risks in your app. <https://developer.android.com/privacy-and-security/risks>
- Google (2024i) Permissions on android. <https://developer.android.com/guide/topics/permissions/overview>
- Google (2024j) Privacy & security SC-DF2. <https://developer.android.com/docs/quality-guidelines/core-app-quality#SC-10>
- Google (2024k) Privacy & security SC-DF3. <https://developer.android.com/docs/quality-guidelines/core-app-quality#sc>
- Google (2024l) Request permissions. <https://developer.android.com/privacy-and-security/about#permissions>
- Google (2024m) SharedPreferences. <https://developer.android.com/privacy-and-security/security-best-practices#sharedpreferences>
- Google (2024n) shouldOverrideUrlLoading. [https://developer.android.com/reference/android/webkit/WebViewClient#shouldOverrideUrlLoading\(android.webkit.WebView,%20android.webkit.WebResourceRequest\)](https://developer.android.com/reference/android/webkit/WebViewClient#shouldOverrideUrlLoading(android.webkit.WebView,%20android.webkit.WebResourceRequest))
- Google (2024o) Store data in external storage on use case. <https://developer.android.com/privacy-and-security/security-best-practices#external-storage>
- Google (2024p) Store private data within internal storage. <https://developer.android.com/privacy-and-security/security-best-practices#internal-storage>
- Google (2024q) WebView objects. <https://developer.android.com/privacy-and-security/security-best-practices#webview>
- JoMingyu (2024) Google-play-scraper. <https://github.com/JoMingyu/google-play-scraper>
- Li L, Gao J, Bissyandé TF, Ma L, Xia X, Klein J (2020) CDA: characterising deprecated android APIs. *Empir Softw Eng* 25(3):2058–2098
- Li Z, Machiry A, Chen B, Naik M, Wang K, Song L (2021) ARBITRAR: user-guided api misuse detection. In: Proceedings of the 42nd IEEE symposium on security and privacy (IEEE S&P), San Francisco, CA, USA, 24–27 May 2021. <https://doi.org/10.1109/SP40001.2021.00090>
- Lin YC (2015) AndroBugs framework. https://github.com/AndroBugs/AndroBugs_Framework#androbugs-framework
- Luo T, Wu J, Yang M, Zhao S, Wu Y, Wang Y (2018) MAD-API: detection, correction and explanation of API misuses in distributed android applications. In: Proceedings of the 7th international conference on artificial intelligence and mobile services (AIMS), Seattle, WA, USA, June 25–30, 201. https://doi.org/10.1007/978-3-319-94361-9_10
- Lyons A, Gamba J, Shawaga A, Reardon J, Tapiador J, Egelman S, Vallina-Rodriguez N (2023) Log: it's big, it's heavy, it's filled with personal data! measuring the logging of sensitive information in the android ecosystem. In: Proceedings of the 32nd USENIX security symposium (USENIX-Sec), Anaheim, CA, USA, August 9–11, 2023. <https://www.usenix.org/conference/usenixsecurity23/presentation/lyons>
- MITRE (2013) Improper export of android application components. <https://cwe.mitre.org/data/definitions/926.html>
- Oltrogge M, Huaman N, Amft S, Acar Y, Backes M, Fahl S (2021) Why eve and mallory still love android: revisiting TLS (In) security in android applications. In: Proceedings of the 30th USENIX security symposium (USENIX-Sec), August 11–13, 2021. <https://www.usenix.org/conference/usenixsecurity21/presentation/oltrogge>
- Overflow S (2015) Android get serial number. <https://stackoverflow.com/questions/33079734/android-get-serial-number>
- OWASP (2024) The mobile application security verification standard. <https://mas.owasp.org/MASVS/>
- Shao Y, Ott J, Jia YJ, Qian Z, Mao ZM (2016) The misuse of android unix domain sockets and security implications. In: Proceedings of the 23rd ACM SIGSAC conference on computer and communications security (CCS), Vienna, Austria, October 24–28, 201. <https://doi.org/10.1145/2976749.2978297>
- Simplilearn (2022) Android developer salary - for freshers and experienced. <https://www.simplilearn.com/tutorials/software-career-resources/android-developer-salary>
- Stack-Overflow (2010a) Remove debug logging. <https://stackoverflow.com/questions/2446248/>

- Stack-Overflow (2010b) Uzip file. <https://stackoverflow.com/questions/3382996/how-to-unzip-files-programmatically-in-android>
- Stack-Overflow (2012a) Android.os.build.serial. <https://stackoverflow.com/questions/11794749/how-to-decode-android-os-build-serial>
- Stack-Overflow (2012b) Share file. <https://stackoverflow.com/questions/12170386/create-and-share-a-file-from-internal-storage>
- Turner A (2024) Google Play App Statistics. <https://www.bankmycell.com/blog/number-of-google-play-store-apps/>
- Vásquez ML, Bavota G, Bernal-Cárdenas C, Penta MD, Oliveto R, Poshyvanyk D (2013) API change and fault proneness: a threat to the success of android apps. In: Proceedings of the 9th joint meeting of the european software engineering conference and the ACM SIGSOFT symposium on the foundations of software engineering (ESEC/FSE), Saint Petersburg, Russian Federation, August 18-26, 2011. <https://doi.org/10.1145/2491411.2491428>
- Vásquez ML, Bavota G, Penta MD, Oliveto R, Poshyvanyk D (2014) How do API changes trigger stack overflow discussions? a study on the android SDK. In: Proceedings of the 22nd international conference on program comprehension (ICPC), Hyderabad, India, June 2-3, 2011. <https://doi.org/10.1145/2597008.2597155>
- Yang S, Hou Q, Li S, Diao W (2023) Do app developers follow the android official data security guidelines? – an empirical measurement on app data security. In: Proceedings of the 30th asia-pacific software engineering conference (APSEC), Seoul, Korea, December 4-7, 2023. <https://doi.org/10.1109/APSEC60848.2023.00017>
- Yang S, Li R, Chen J, Diao W, Guo S (2022) Demystifying android non-SDK APIs: measurement and understanding. In: Proceedings of the 44th IEEE/ACM international conference on software engineering (ICSE), , Pittsburgh, PA, USA, May 25-27, 2022. <https://doi.org/10.1145/3510003.3510045>
- Zhang T, Upadhyaya G, Reinhardt A, Rajan H, Kim M (2018) Are code examples on an online Q&A forum reliable? a study of API misuse on stack overflow. In: Proceedings of the 40th international conference on software engineering (ICSE), Gothenburg, Sweden, May 27 - June 03, 2018. <https://doi.org/10.1145/3180155.3180260>
- Zhou R, Hamdaqa M, Cai H, Hamou-Lhadj A (2020) MobiLogLeak: a preliminary study on data leakage caused by poor logging practices. In: Proceedings of the 27th IEEE international conference on software analysis, evolution and reengineering (SANER), London, ON, Canada, February 18-21, 2020. <https://doi.org/10.1109/SANER48275.2020.9054831>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

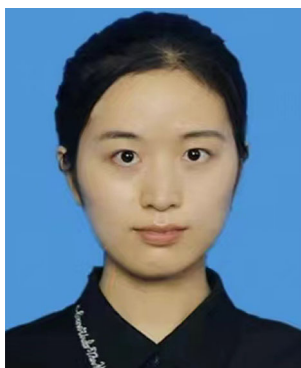
Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Shishuai Yang received the B.Eng. degree in Software Engineering from Henan University in 2020. He is currently pursuing the Ph.D. degree at School of Cyber Science and Technology, Shandong University, Qingdao, China. His research interests are mobile security and mobile measurement.



Qinsheng Hou received the Ph.D. degree in School of Cyber Science and Technology from Shandong University in 2024. His research interests are mobile security, software supply chain security, and IoT security.



Shuang Li received the B.Sc. degree in Information Security from Shandong University, Qingdao, China, in 2021. She is currently working toward the Ph.D. degree at the School of Cyber Science and Technology, Shandong University, Qingdao, China. Her research interests include mobile security and software security.



Fenghao Xu is currently a research associate professor at School of Cyber Science and Engineering, Southeast University, China. Before join SEU, he has obtained his PhD degree in information engineering at the Chinese University of Hong Kong, under the supervision of Prof. Kehuan Zhang. He has visited University of California, Irvine as a research scholar in 2020. His primary research interest lies in network and system security, with a special focus on mobile and IoT security.



Wenrui Diao received the Ph.D. degree in Information Engineering from The Chinese University of Hong Kong in 2017. He is currently a professor with the School of Cyber Science and Technology, Shandong University, Qingdao, China. His research interests include mobile security and IoT security. He received the 2019 ACM SIGSAC China Rising Star Award.